



Pertemuan 4

▼ Data JSON (List)



Persiapkan folder dan file html untuk `pertemuan-4` lengkap dengan `Tailwind CSS` .

Sekarang kita akan coba menampilkan data yang berasal dari JSON.

Buatlah sebuah file di `/src/pertemuan_4/framework.json` dengan data sebagai berikut:

▼ *framework.json*

```
[
  {
    "id": 1,
    "name": "React",
    "description": "A JavaScript library for building user interfaces.",
    "details": {
      "developer": "Meta (Facebook)",
      "releaseYear": 2013,
      "officialWebsite": "https://react.dev/"
    },
    "tags": ["JavaScript", "Library", "UI", "Component-Based"]
  },
  {
    "id": 2,
    "name": "Vue",
```

```

    "description": "The Progressive JavaScript Framework.",
    "details": {
      "developer": "Evan You",
      "releaseYear": 2014,
      "officialWebsite": "https://vuejs.org/"
    },
    "tags": ["JavaScript", "Framework", "Reactive", "Lightweight"]
  },
  {
    "id": 3,
    "name": "Angular",
    "description": "One framework. Mobile & desktop.",
    "details": {
      "developer": "Google",
      "releaseYear": 2010,
      "officialWebsite": "https://angular.io/"
    },
    "tags": ["JavaScript", "Framework", "TypeScript", "Enterprise"]
  },
  {
    "id": 4,
    "name": "Svelte",
    "description": "Cybernetically enhanced web apps.",
    "details": {
      "developer": "Rich Harris",
      "releaseYear": 2016,
      "officialWebsite": "https://svelte.dev/"
    },
    "tags": ["JavaScript", "Compiler", "Reactive", "Lightweight"]
  },
  {
    "id": 5,

```

```

    "name": "Next.js",
    "description": "The React Framework for Production.",
    "details": {
      "developer": "Vercel",
      "releaseYear": 2016,
      "officialWebsite": "https://nextjs.org/"
    },
    "tags": ["React", "Framework", "SSR", "Static Generation"],
  },
  {
    "id": 6,
    "name": "Nuxt.js",
    "description": "The Intuitive Vue Framework.",
    "details": {
      "developer": "Nuxt Labs",
      "releaseYear": 2016,
      "officialWebsite": "https://nuxt.com/"
    },
    "tags": ["Vue", "Framework", "SSR", "Static Generation"],
  },
  {
    "id": 7,
    "name": "Gatsby",
    "description": "Fast in every way that matters.",
    "details": {
      "developer": "Gatsby Inc.",
      "releaseYear": 2015,
      "officialWebsite": "https://www.gatsbyjs.com/"
    },
    "tags": ["React", "Static Site", "SEO", "GraphQL"],
  },
  {
    "id": 8,

```

```

    "name": "Ember",
    "description": "A framework for ambitious web developers.",
    "details": {
        "developer": "Ember Core Team",
        "releaseYear": 2011,
        "officialWebsite": "https://emberjs.com/"
    },
    "tags": ["JavaScript", "Framework", "Convention over Configuration"]
},
{
    "id": 9,
    "name": "Meteor",
    "description": "Build apps fast, simple, scalable.",
    "details": {
        "developer": "Meteor Software",
        "releaseYear": 2012,
        "officialWebsite": "https://www.meteor.com/"
    },
    "tags": ["Full-Stack", "JavaScript", "Realtime", "Web & Mobile"]
},
{
    "id": 10,
    "name": "Backbone.js",
    "description": "Give your JS app some Backbone.",
    "details": {
        "developer": "Jeremy Ashkenas",
        "releaseYear": 2010,
        "officialWebsite": "https://backbonejs.org/"
    },
    "tags": ["JavaScript", "Library", "MVC", "Lightweight"]
}

```

```
}  
]
```

Buat sebuah komponen `FrameworkList` yang mengambil data dari json

```
import frameworkData from "./framework.json";  
  
export default function FrameworkList() {  
  return (  
    <div className="p-8">  
      {frameworkData.map  
((item) => (  
        <div key={item.id} className="border p-4 mb-4 rounded-lg shadow-md bg-white">  
          <h2 className="text-lg font-bold text-gray-800">{item.name}</h2>  
          <p className="text-gray-600">{item.description}</p>  
        </div>  
      ) ) }  
    </div>  
  )  
}
```

React

A JavaScript library for building user interfaces.

Vue

The Progressive JavaScript Framework.

Angular

- File JSON yang terletak di `/src/pertemuan_4/framework.json` dapat diakses melalui `import` :

```
import frameworkData from "./framework.json";
```

Jika file terletak di `/public` maka butuh `fetch()` untuk mengakesnya

- `frameworkData` dapat diakses menggunakan `map()` di javascript untuk menampilkan setiap item dalam array

```
{frameworkData.map((item) => (  
  <div key={item.id} className="border p-4 mb-4 rounded  
  lg shadow-md bg-white">  
    <h2 className="text-lg font-bold text-gray-80  
0">{item.name}</h2>  
    <p className="text-gray-600">{item.description}  
</p>  
  </div>  
))}
```



Perhatikan perbedaan data JSON berikut:

Jika data JSON `frameworkData` diawali dengan array (`[]`) maka pemanggilan item nya bisa langsung dilakukan setelah `map()`

```
[
  { "id": 1, "name": "React", "description": "A JavaScript library for building user interfaces." },
  { "id": 2, "name": "Vue", "description": "The Progressive JavaScript Framework." }
]
```

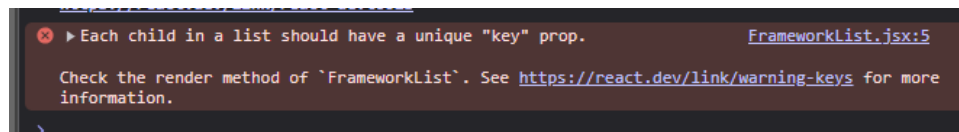
```
{FrameworkData.map((item) => (
  <div key={item.id}>
    <h2>{item.name}</h2>
    <p>{item.description}</p>
  </div>
))}
```

Namun, jika data JSON `frameworkData` diawali dengan object (`{}`) maka kita harus mengakses array-nya terlebih dahulu sebelum memanggil `map()` dan mengakses item-nya:

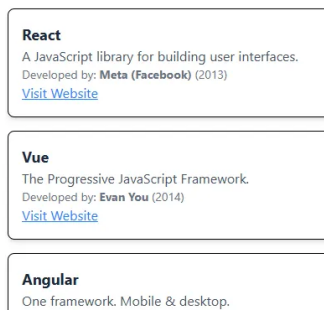
```
{
  "frameworks": [
    { "id": 1, "name": "React", "description": "A JavaScript library for building user interfaces." },
    { "id": 2, "name": "Vue", "description": "The Progressive JavaScript Framework." }
  ]
}
```

```
{FrameworkData.frameworks.map((item) => (
  <div key={item.id}>
    <h2>{item.name}</h2>
    <p>{item.description}</p>
  </div>
))}
```

- Setiap komponen yang dihasilkan melalui `map()` harus memiliki `key` sebagai pembeda antar komponen dan harus unique. `Key` ini diperlukan oleh React untuk mengoptimalkan rendering dan menghindari re-rendering yang tidak perlu. Jika komponen tersebut tidak memiliki `key`, maka akan muncul warning seperti berikut:



Lengkapi komponen `FrameworkList` dengan menampilkan informasi `officialWebsite`, `developer`



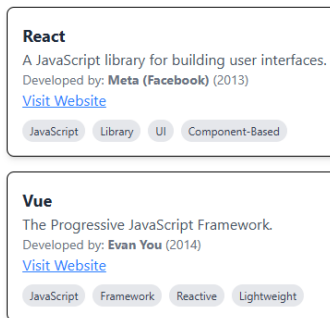
Pemanggilan data yang berada dalam **nested structured** dapat diakses dengan cara mengakses properti dalam objek menggunakan dot notation (`.`)

```
items.detail.officialWebsite
```

```
items.detail.developer
```

(Silahkan tambahkan di project masing-masing)

Lengkapi komponen dengan menampilkan informasi `tags`



(Silahkan custom warna tags sesuai keinginan)

Data `tags` yang ada di setiap `item` merupakan data array, sehingga untuk mengaksesnya juga menggunakan `map()` sehingga pemanggilannya dilakukan sebagai berikut

```
item.tags.map(tag,index)
```

```
{item.tags.map((tag,index)=>(
  <span key={index} className="bg-
gray-200 text-gray-700 px-2 py-1 tex
t-xs rounded-full mr-2">
    {tag}
  </span>
))}
```

▼ Data JSON (Search & Filter)

Kita sudah berhasil menampilkan list data dari JSON. Sekarang kita akan menambahkan fitur search dan filter agar komponen menjadi lebih interaktif.

- Duplikat komponen `FrameworkList` menjadi `FrameworkListSearchFilter` dan panggil komponen tersebut di `main.jsx` .


```

7   createRoot(document.getElementById("root"))
8   .render(
9     <div>
10      { /* <FrameworkList/> */ }
11      <FrameworkListSearchFilter/>
12    </div>
13  )
14

```

- Tambahkan inputan search dan select untuk filter dengan data kosong

```

...
<input
  type="text"
  name="searchTerm"
  placeholder="Search f
ramework..."
  className="w-full p-2
border border-gray-300
rounded mb-4"
/>

<select
  name="selectedTag"
  className="w-full p-2
border border-gray-300
rounded mb-4"
>
  <option value="">All
Tags</option>
</select>

{filteredFrameworks.map
((item, index) => (
  ...
))}

```

React

A JavaScript library for building user interfaces.
 Developed by: **Meta (Facebook)** (2013)
[Visit Website](#)

JavaScript
Library
UI
Component-Based

- Inisialisasi `state` untuk menyimpan value dari inputan yang tersedia.

```
export default function FrameworkList() {
  const [searchTerm, setSearchTerm] = useState
  ("");
  const [selectedTag, setSelectedTag] = useState
  ("");
  ...
  return (
    ...
  )
}
```

- Terapkan logic dalam memfilter data JSON sesuai dengan `searchTerm` dan `selectedTag`

```
/** Deklarasi state */
...

const _searchTerm = searchTerm.toLowerCase();
const filteredFrameworks = frameworkData.filter((framework) => {
  const matchesSearch =
    framework.name
      .toLowerCase()
      .includes(_searchTerm) ||
    framework.description
      .toLowerCase()
      .includes(_searchTerm);

  const matchesTag = selectedTag ? framework.tags.includes(selectedTag) : true;

  return matchesSearch && matchesTag;
});
...

```

```

    return (
      ...
    )

```

- Terapkan logic untuk mengambil nilai unique dari list tags yang ada di JSON dan memasukkannya ke `allTags`

```

/** Deklarasi state */
...

/** Deklarasi Logic Search & Filter */
...

/** Deklarasi pengambilan unique tags di frameworkData */
**/
const allTags = [
  ...new Set(
    frameworkData.flatMap((framework) => framework.tags)
  ),
];

...

return (
  ...
)

```

▼ Penjelasan Singkat

1. `flatMap((framework) => framework.tags)`
 - Mengambil semua `tags` dari setiap `framework` dalam `frameworkData` dan menggabungkannya menjadi satu array datar.
 - Misalnya, jika ada beberapa `framework` dengan tags seperti:

```

[
  { name: "React", tags: ["UI", "Frontend"] },
  { name: "Vue", tags: ["UI", "Frontend"] },
  { name: "Node.js", tags: ["Backend", "JavaScr

```

```
ipt"] }  
]
```

Maka hasil dari `flatMap` adalah:

```
["UI", "Frontend", "UI", "Frontend", "Backend",  
"JavaScript"]
```

2. `new Set(...)`

- Mengubah array menjadi **Set**, yaitu struktur data yang otomatis menghilangkan duplikasi.
- Hasilnya:

```
Set { "UI", "Frontend", "Backend", "JavaScript"  
}
```

3. `[...new Set(...)]`

- Mengubah `Set` kembali menjadi array dengan spread operator (`[...]`).
- Hasil akhirnya:

```
["UI", "Frontend", "Backend", "JavaScript"]
```

Dari penjelasan diatas, variabel `allTags` akan menjadi sebuah array, sehingga kita bisa menerapkan `map()` untuk menampilkannya di tag `<option>` seperti berikut:

```
<option value="">All Tags</option>  
  {allTags.map((tag, index) => (  
    <option key={index} value={tag}>  
      {tag}
```

```

    </option>
  )})

```

- Terapkan `onChange` pada input dan selected untuk mengupdate masing-masing state dan tes apakah Search & Filter berhasil

```

<input
  ...
  onChange={(e) => setSearchTerm(e.target.value)}
/>

```

```

<select
  ...
  onChange={(e) => setSelectedTag(e.target.value)}
>

```

- Terakhir, pastikan mengganti `frameworkData` menjadi `filteredFrameworks` agar data yang di mapping menggunakan data yang telah di filter

Sebelum

```

{frameworkData.map((item, index) => (
  <div ...
  </div>
))}

```

Sesudah

```

{filteredFrameworks.map((item, index) => (
  <div ...
  </div>
))}

```

Silahkan tes dengan mengetikkan keyword di kolom search dan memilih tag yang tersedia.

Logic yang kita terapkan ialah mencari `name` dan `description` yang cocok dengan keyword lalu memilih `tags` yang sesuai dengan list tag yang tersedia

▼ Best Practice State

Jika kita memiliki 1 inputan dan 1 select seperti pada pembahasan sebelumnya, kita akan melakukan langkah berikut untuk menyiapkan state-nya:

1. Inisialisasi state

```
const [searchTerm, setSearchTerm] = useState("");
const [selectedTag, setSelectedTag] = useState("");
```

2. Terapkan `onChange` ke masing-masing state

```
<input
  ...
  onChange={(e) => setSearchTerm(e.target.value)}
/>
```

```
<select
  ...
  onChange={(e) => setSelectedTag(e.target.value)}
>
```

3. `searchTerm` dan `selectedTag` siap digunakan misal menerapkan `toLowerCase()`

```
const _searchTerm = searchTerm.toLowerCase();
const _selectedTag = selectedTag.toLowerCase();
```

Hal ini tidak masalah jika inputan tidak banyak. Namun bagaimana jika kita memiliki form yang memiliki input, select, checkbox dll dalam jumlah yang tidak sedikit? tentu ini akan sangat tidak efisien.

Karena itu kita akan update cara pemanggilan state dan `onChange`-nya agar lebih general dan bisa diterapkan di berbagai macam form

1. Inisialisasi state `dataForm` beserta function `handleChange` sebagai berikut:

```
export default function FrameworkList() {
  //Deklarasi state sebelumnya silahkan di commen
  t atau dihapus
  ...

  /*Inisialisasi DataForm*/
```

```

const [dataForm, setDataForm] = useState({
  searchTerm: "",
  selectedTag: "",
  /*Tambah state lain beserta default value*/
});

/*Inisialisasi Handle perubahan nilai input for
m*/

const handleChange = (evt) => {
  const { name, value } = evt.target;
  setDataForm({
    ...dataForm,
    [name]: value,
  });
};

```



2. Sesuaikan `onchange` pada setiap input dan tambahkan property `name` seperti berikut:

Before

```

<input
  ...
  onChange={(e) => setSearchTerm(e.target.value)}
/>

```

After

```

<input
  ...
  name="searchTerm"
  onChange={handleChange}
/>

```

Before

```

<select
  ...
  onChange={(e) => setSelectedTag(e.target.value)}
>

```

After

```
<select
  name="selectedTag"
  onChange={handleChange}
>
```

- Ubah pemanggilan state `searchTerm` dan `selectedTag` yang sudah ada menjadi `dataForm.searchTerm` dan `dataForm.selectedTag`

▼ Responsive & Grid Design

Responsive design adalah teknik desain web agar tampilan website bisa menyesuaikan berbagai ukuran layar, seperti **mobile, tablet, dan desktop**.

Tailwind mendukung responsive design dengan menerapkan breakpoint yang sudah ditetapkan:

Breakpoint	Kelas Prefix	Lebar Minimum
<code>sm</code>	<code>sm:</code>	<code>640px</code> (Tablet Kecil)
<code>md</code>	<code>md:</code>	<code>768px</code> (Tablet Besar)
<code>lg</code>	<code>lg:</code>	<code>1024px</code> (Laptop Kecil)
<code>xl</code>	<code>xl:</code>	<code>1280px</code> (Desktop)
<code>2xl</code>	<code>2xl:</code>	<code>1536px</code> (Layar Besar)

Jika tidak ada prefix, maka berlaku untuk semua ukuran layar kecil (default: mobile).

Buatlah component parent `ResponsiveDesign` dan panggil di `main.jsx`.

Silahkan terapkan beberapa child component berikut:

- Ukuran Text akan menyesuaikan lebar layar:


```
function ResponsiveText(){
  return (
    <p className="text-sm md:text-lg lg:text-xl xl:
text-2xl mb-4">
      Coba lakukan zoom in atau zoom out. Perhati
      kan bahwa ukuran teks akan menyesuaikan dengan ukuran l
      ayar.<br/>
      Coba hapus class yang menggunakan prefix br
      eakpoint (md:xxx, lg:xxx, xl:xxx) dan lihat perbedaanny
      a!
    </p>
  )
}
```

- Width sebuah komponen akan mengikuti setiap class yang ditetapkan pada breakpoint tertentu

```
function ResponsiveWidth(){
  return (
    <div className="mb-4">
      <p>
        Coba lakukan **zoom in/zoom out** atau
        ubah ukuran layar. Perhatikan bagaimana posisi kolom ak
        an menyesuaikan secara responsif:<br/> <br/>
      </p>
      <p>
        <strong>md:w-1/2</strong> → Saat layar
        mencapai ukuran tablet (md: 768px) atau lebih besar, se
        tiap kolom akan memiliki lebar 50%.
      </p>
      <p>
        <strong>md:flex-row</strong> pada div p
        arent membuat dua kolom ini sejajar secara horizontal p
```

ada layar tablet ke atas,

sedangkan pada layar lebih kecil (mobile), kolom akan tersusun vertikal secara default (`flex-col`).

</p>

```
<div className="flex flex-col md:flex-row mb-4">
  <div className="bg-red-400 w-full md:w-1/2 p-4">Kolom 1</div>
  <div className="bg-blue-400 w-full md:w-1/2 p-4">Kolom 2</div>
</div>
)
```

- Penggunaan Grid akan dalam membagi tampilan layar menjadi beberapa kolom

```
function ResponsiveLayout(){
  return (
    <div>
```

```
      <p class="mt-4">
```

Kotak-kotak di bawah ini menggunakan Grid Layout dari Tailwind CSS. Jumlah kolom akan menyesuaikan dengan ukuran layar:

```
      </p>
```

```
      <p>
```

- grid-cols-1 → Pada layar kecil (default), semua box tersusun dalam 1 kolom.

- sm:grid-cols-2 → Saat ukuran layar mencapai sm (≥640px), g

```

rid berubah menjadi <strong>2 kolom</strong>.<br/>
    - <strong>md:grid-cols-3</strong> → Pada ukuran <strong>md (≥768px)</strong>, grid berubah menjadi <strong>3 kolom</strong>.<br/>
    - <strong>lg:grid-cols-4</strong> → Saat ukuran layar <strong>lg (≥1024px)</strong> atau lebih besar, grid akan memiliki <strong>4 kolom</strong>.<br/>
>
    </p>
    <div class="grid grid-cols-1 sm:grid-cols-2 md:grid-cols-3 lg:grid-cols-4 gap-4 mt-4">
        <div class="bg-blue-500 p-4">Box 1</div>
    >
        <div class="bg-blue-500 p-4">Box 2</div>
    >
        <div class="bg-blue-500 p-4">Box 3</div>
    >
        <div class="bg-blue-500 p-4">Box 4</div>
    >
    </div>
</div>
)
}

```



Perbedaan antara Tailwind CSS dengan Bootstrap saat membagi kolom:

Bootstrap

```
<div class="container">
  <div class="row">
    <div class="col-md-6 bg-primary text-white p-3">Kolom 1 (50%)</div>
    <div class="col-md-6 bg-secondary text-white p-3">Kolom 2 (50%)</div>
  </div>
</div>
```

TailwindCSS

```
<div class="grid grid-cols-1 md:grid-cols-2 gap-4">
  <div class="bg-blue-500 text-white p-4">Kolom 1 (50%)</div>
  <div class="bg-green-500 text-white p-4">Kolom 2 (50%)</div>
</div>
```

Fitur	Bootstrap (<code>col-md</code>)	Tailwind (<code>grid-cols</code>)
Sistem Layout	Flexbox (12 kolom)	CSS Grid
Penulisan Struktur	<code>row</code> + <code>col-xx-*</code>	<code>grid</code> + <code>grid-cols-*</code>
Responsiveness	Kolom bisa berubah ukuran berdasarkan kelas <code>col-xx-*</code>	Kolom bisa bertambah atau berkurang sesuai <code>grid-cols-*</code>
Default Behavior	Jika tidak cukup ruang, kolom akan turun ke bawah	Grid tetap dalam jumlah kolom yang ditentukan

Penyesuaian	Terbatas pada sistem 12 kolom	Fleksibel, bisa <code>grid-cols-3</code> , <code>grid-cols-7</code> , dll.
--------------------	-------------------------------	--
